# Billing Program for a Rural Electricity Co-operative

## FILE 2 of 2 : SOURCE CODE, INPUT FILES, EXECUTION OUTPUT AND README

| | |
|---|---|
| Student | R. Surya |
| Register No. | 1924260114C |
| Platform | gcc 13.3.0, Ubuntu 24.04, x86-64 (all outputs below were produced by actually compiling and running the code) |
| Companion file | FILE 1 : Assignment report (analysis, UML, pseudocode, test tables, complexity analyses for Tasks 1-12) |

### Contents


- Section A - README.md
- Section B - Core program (Tasks 1-3, 9): tariff.h, billing.c, main.c
- Section C - Input and configuration files
- Section D - Task programs (Tasks 2-7)
- Section E - Task 8 files and script
- Section F - Tests, scale test and smart meter (Tasks 9, 10, 12)
- Section G - Execution output for every task (evidence)
- Section H - Build scripts


Submission requirements covered here: executed code for every task, all completed code exercises, input files, output (as captured terminal text) and README.md. For the GitHub repository, copy the folder structure shown in the README, add your own terminal screenshots (run ./run_all.sh and ./run_task8.sh) and push it; File 1 holds the report evidence for Tasks 1-12.

# Section A - README.md

```
# Rural Electricity Co-operative - Billing Program (C)

Student: R. Surya  |  Register No.: 1924260114C  |  SIMATS Engineering - C Programming Lab, Assignment 1, Q1

## Project layout
| File | Purpose | Task |
|------|---------|------|
| tariff.h | constants, macros, enums, structs, include guard, DBG(), prototypes | 3,4,5 |
| billing.c | parse_reading, compute_units, slab charges (if-else + array), generate_bill | 1-3,9 |
| main.c | batch driver (stdin) | 9 |
| task2_types.c | sizeof, limits.h/float.h, float drift, 16-bit rollover, double vs paise | 2 |
| task3_const.c | #define vs const vs enum; RATE2 = 230 | 3 |
| task4_macros.c, macro_expand.c | macro safety, gcc -E, macro vs inline timing | 4 |
| task5_debug.c, noguard.h, noguard_demo.c | include guard, -DDEBUG, <> vs "" | 5 |
| task6_io.c, task6_wrong.c, task6_compare.c | scanf/printf, wrong specifiers, scanf vs strtol | 6 |
| task7_conv.c | conversions and precision | 7 |
| run_task8.sh, err_*.c, warn.c, warn_fixed.c | compilation stages, errors, warnings | 8 |
| test_slabs.c, inputs/tests_input.txt, inputs/five_bills.txt | testing, slabs vs array | 9 |
| task10_scale.c | float/int/long/long long at 4,000 / 60,000 / 800,000 bills | 10 |
| smart_meter.c, tariff.cfg, inputs/readings_day.csv | smart-meter redesign | 12 |
| run_all.sh | compiles and runs everything; results in outputs/ | all |

## Build and run
    gcc -Wall -Wextra -o billing main.c billing.c
    ./billing < inputs/tests_input.txt
    gcc -Wall -Wextra -DRATE2_PAISE=230 -o billing_r230 main.c billing.c   # Task 3 tariff change
    gcc -Wall -Wextra -DDEBUG -o t5_debug task5_debug.c billing.c          # Task 5 debugging
    gcc -Wall -Wextra -o smart_meter smart_meter.c && ./smart_meter tariff.cfg   # Task 12
    ./run_all.sh && ./run_task8.sh                                          # everything

## Assumptions
Telescopic slabs 100 @ 150 paise, next 100 @ 200 paise, rest @ 300 paise; fixed charge 5000/15000/2500 paise
(Domestic/Commercial/Agriculture); all money in integer paise; 16-bit unsigned meter register; district = 800,000 households.
Tested with gcc 13.3 on Ubuntu 24.04 (LP64).  On Windows sizeof(long) = 4.
```

# Section B - Core program (Tasks 1-3, 9)

### tariff.h - constants, macros, enums, structs, include guard, DBG()

```
/* tariff.h - constants, macros, types and prototypes (Tasks 3,4,5)  */
#ifndef TARIFF_H            /* include guard: prevents double inclusion */
#define TARIFF_H

#include <stdio.h>

/* ---- Tariff constants (all money in PAISE; 100 paise = 1 rupee) ---- */
#ifndef RATE1_PAISE         /* overridable: gcc -DRATE2_PAISE=230       */
#define RATE1_PAISE 150L    /* units   1..100  : Rs 1.50 / unit         */
#endif
#ifndef RATE2_PAISE
#define RATE2_PAISE 200L    /* units 101..200  : Rs 2.00 / unit         */
#endif
#ifndef RATE3_PAISE
#define RATE3_PAISE 300L    /* units  >200     : Rs 3.00 / unit         */
#endif
#define SLAB1_LIMIT     100L
#define SLAB2_LIMIT     200L
#define METER_MAX       65535L      /* 16-bit unsigned register max      */
#define METER_MODULUS   65536L      /* register wraps modulo 2^16        */
#define MAX_HOUSEHOLD_NO 999999L

/* enum constant set: customer categories (type-safe-ish, debugger-visible) */
enum Category { CAT_DOMESTIC = 0, CAT_COMMERCIAL = 1, CAT_AGRICULTURE = 2, CAT_COUNT };

/* const objects (defined in billing.c) */
extern const long FIXED_CHARGE_PAISE[CAT_COUNT];
extern const char *const CATEGORY_NAME[CAT_COUNT];

/* ---- Safe macros ---- */
#define RUPEES_TO_PAISE(r)  ((long)((r) * 100.0 + 0.5))
#define MAX(a, b)           (((a) > (b)) ? (a) : (b))
#define SQUARE(x)           ((x) * (x))

/* ---- Conditional debugging (compile with gcc -DDEBUG) ---- */
#ifdef DEBUG
#define DBG(fmt, ...) fprintf(stderr, "[DEBUG] %s:%d: " fmt "\n", __FILE__, __LINE__, ##__VA_ARGS__)
#else
#define DBG(fmt, ...) ((void)0)
#endif

/* ---- Data types ---- */
typedef struct {
    long           household_no;
    int            category;       /* enum Category */
    unsigned short prev, curr;     /* 16-bit meter register values */
    int            rollover;       /* 1 = meter wrapped past 65535 */
} Customer;

typedef struct {
    long units;
    long energy_paise;
    long fixed_paise;
    long total_paise;
} Bill;

/* ---- Error codes ---- */
enum BillStatus { BILL_OK = 0, ERR_FORMAT, ERR_NEGATIVE, ERR_RANGE, ERR_DECREASE,
                  ERR_ROLLOVER_FLAG, ERR_CATEGORY, ERR_HOUSEHOLD };

/* ---- Prototypes ---- */
int         parse_reading(const char *s, unsigned short *out);
int         compute_units(unsigned short prev, unsigned short curr, int rollover, long *units);
long        energy_charge_paise(long units);          /* if-else slabs   */
long        energy_charge_paise_array(long units);    /* array + loop    */
int         generate_bill(const Customer *c, Bill *b);
const char *status_text(int status);
void        print_money(long paise);

#endif /* TARIFF_H */
```

### billing.c - validation, rollover, slab charge (if-else and array versions), bill generation

```
/* billing.c - core billing logic */
#include <stdio.h>
#include <stdlib.h>
#include <ctype.h>
#include <errno.h>
#include <limits.h>
#include "tariff.h"

const long FIXED_CHARGE_PAISE[CAT_COUNT] = { 5000L, 15000L, 2500L };   /* Rs 50 / 150 / 25 */
const char *const CATEGORY_NAME[CAT_COUNT] = { "Domestic", "Commercial", "Agriculture" };

/* Validate & convert a text reading to 16-bit register value */
int parse_reading(const char *s, unsigned short *out)
{
    const char *p = s;
    char *end;
    long v;
    if (s == NULL || *s == '\0') return ERR_FORMAT;
```

```
    if (*p == '-') {                        /* negative number?          */
        for (++p; *p; ++p) if (!isdigit((unsigned char)*p)) return ERR_FORMAT;
        return (p == s + 1) ? ERR_FORMAT : ERR_NEGATIVE;
    }
    for (; *p; ++p) if (!isdigit((unsigned char)*p)) return ERR_FORMAT;  /* letters etc. */
    errno = 0;
    v = strtol(s, &end, 10);
    if (errno == ERANGE || v > METER_MAX) return ERR_RANGE;
    *out = (unsigned short)v;
    return BILL_OK;
}

/* units consumed, handling 16-bit rollover */
int compute_units(unsigned short prev, unsigned short curr, int rollover, long *units)
{
    if (rollover) {
        if (curr >= prev) return ERR_ROLLOVER_FLAG;       /* cannot have wrapped */
        *units = METER_MODULUS - prev + curr;
    } else {
        if (curr < prev) return ERR_DECREASE;             /* reading went down   */
        *units = (long)curr - prev;
    }
    return BILL_OK;
}

/* telescopic slabs using macros (if-else version) */
long energy_charge_paise(long units)
{
    if (units <= SLAB1_LIMIT)
        return units * RATE1_PAISE;
    else if (units <= SLAB2_LIMIT)
        return SLAB1_LIMIT * RATE1_PAISE + (units - SLAB1_LIMIT) * RATE2_PAISE;
    else
        return SLAB1_LIMIT * RATE1_PAISE
             + (SLAB2_LIMIT - SLAB1_LIMIT) * RATE2_PAISE
             + (units - SLAB2_LIMIT) * RATE3_PAISE;
}

/* same tariff, table-driven (array + loop) */
long energy_charge_paise_array(long units)
{
    static const struct { long upto; long rate; } slab[] = {
        { SLAB1_LIMIT, RATE1_PAISE }, { SLAB2_LIMIT, RATE2_PAISE }, { LONG_MAX, RATE3_PAISE }
    };
    long total = 0, lower = 0;
    for (size_t i = 0; i < sizeof slab / sizeof slab[0] && units > lower; ++i) {
        long top = (units < slab[i].upto) ? units : slab[i].upto;
        total += (top - lower) * slab[i].rate;
        lower = slab[i].upto;
    }
    return total;
}

int generate_bill(const Customer *c, Bill *b)
{
    int st;
    if (c->household_no < 1 || c->household_no > MAX_HOUSEHOLD_NO) return ERR_HOUSEHOLD;
    if (c->category < 0 || c->category >= CAT_COUNT)               return ERR_CATEGORY;
    st = compute_units(c->prev, c->curr, c->rollover, &b->units);
    if (st != BILL_OK) return st;
    b->energy_paise = energy_charge_paise(b->units);
    b->fixed_paise  = FIXED_CHARGE_PAISE[c->category];
    b->total_paise  = b->energy_paise + b->fixed_paise;
    DBG("hh=%ld prev=%u curr=%u units=%ld energy=%ld fixed=%ld total=%ld",
        c->household_no, c->prev, c->curr, b->units, b->energy_paise, b->fixed_paise, b->total_paise);
    return BILL_OK;
}

const char *status_text(int s)
{
    switch (s) {
    case BILL_OK:           return "OK";
    case ERR_FORMAT:        return "reading is not a number (letters/symbols)";
    case ERR_NEGATIVE:      return "negative reading";
    case ERR_RANGE:         return "reading outside 0..65535";
    case ERR_DECREASE:      return "current < previous without rollover";
    case ERR_ROLLOVER_FLAG: return "rollover flagged but current >= previous";
    case ERR_CATEGORY:      return "invalid category (0=Dom,1=Com,2=Agri)";
    case ERR_HOUSEHOLD:     return "invalid household number";
    default:                return "unknown error";
    }
}

void print_money(long paise) { printf("Rs %ld.%02ld", paise / 100, paise % 100); }
```

### main.c - batch driver

```
/* main.c - batch billing driver: reads lines "household category prev curr rollover(y/n)" from stdin */
#include <stdio.h>
#include <string.h>
#include "tariff.h"

int main(void)
{
    char line[256], ps[32], cs[32], rs[32];
    long hh; int cat, ok = 0, bad = 0, lineno = 0;
```

```
    long grand = 0;

    printf("%-6s %-5s %-11s %-12s %7s %12s %10s %12s\n",
           "Line", "HH", "Category", "Reading", "Units", "Energy", "Fixed", "Total");
    while (fgets(line, sizeof line, stdin)) {
        Customer c; Bill b; int st;
        ++lineno;
        if (line[0] == '#' || line[0] == '\n') continue;
        if (sscanf(line, "%ld %d %31s %31s %31s", &hh, &cat, ps, cs, rs) != 5) {
            printf("%-6d ERROR: malformed line -> %s", lineno, line); ++bad; continue;
        }
        c.household_no = hh; c.category = cat;
        c.rollover = (rs[0] == 'y' || rs[0] == 'Y');
        if ((st = parse_reading(ps, &c.prev)) != BILL_OK || (st = parse_reading(cs, &c.curr)) != BILL_OK) {
            printf("%-6d ERROR (HH %ld): %s  [prev=%s curr=%s]\n", lineno, hh, status_text(st), ps, cs); ++bad; continue;
        }
        if ((st = generate_bill(&c, &b)) != BILL_OK) {
            printf("%-6d ERROR (HH %ld): %s  [prev=%u curr=%u rollover=%c]\n",
                   lineno, hh, status_text(st), c.prev, c.curr, c.rollover ? 'y' : 'n'); ++bad; continue;
        }
        printf("%-6d %-5ld %-11s %5u->%-6u %7ld ", lineno, hh, CATEGORY_NAME[cat], c.prev, c.curr, b.units);
        printf("%12ld %10ld %12ld   (", b.energy_paise, b.fixed_paise, b.total_paise);
        print_money(b.total_paise); printf(")\n");
        grand += b.total_paise; ++ok;
    }
    printf("\nValid bills: %d   Rejected: %d   Grand total: ", ok, bad);
    print_money(grand); printf("  (%ld paise)\n", grand);
    return 0;
}
```

# Section C - Input and configuration files

### inputs/tests_input.txt

```
# household category prev curr rollover   (amounts printed in paise)
# ---- normal bills across all slabs ----
1001 0 1000 1050 n
1002 0 1000 1150 n
1003 0 1000 1300 n
1004 1  500  620 n
1005 2 2000 2250 n
# ---- boundary cases ----
1006 0  500  600 n
1007 0  500  700 n
1008 0  500  500 n
1009 0  500  601 n
1010 0  500  701 n
# ---- meter rollover ----
1011 0 65535    0 y
1012 0 65500   40 y
1013 0 65400  100 y
# ---- invalid input ----
1014 0 abc 100 n
1015 0 100 12x n
1016 0 -5 100 n
1017 0 1200 1100 n
1018 0 100 70000 n
1019 0 100 200 y
1020 7 100 200 n
bad line here
```

### inputs/five_bills.txt

```
2001 0 1000 1050 n
2002 0 1000 1150 n
2003 0 1000 1300 n
2004 1 500 620 n
2005 2 2000 2250 n
```

### inputs/input_valid.txt

```
1001 12345 600000 8000000000 1.505 2.7182818 D Surya
```

### inputs/input_invalid.txt

```
1001 abc 600000 8000000000 1.505 2.7182818 D Surya
```

### inputs/compare_lines.txt

```
12abc
abc
   42
99999999999
-7
```

### tariff.cfg

```
# Smart-meter tariff configuration (loaded at RUNTIME - no recompilation needed)
# key                       value
SLAB1_LIMIT_KWH             100
SLAB2_LIMIT_KWH             200
RATE1_PAISE_PER_KWH         150
RATE2_PAISE_PER_KWH         200
RATE3_PAISE_PER_KWH         300
PEAK_START_HOUR             18
PEAK_END_HOUR               22
PEAK_SURCHARGE_PERCENT      20
FIXED_CHARGE_PAISE          5000
```

### inputs/readings_day.csv

```
# time,Wh_in_interval  (96 readings/day x 365 = 35040 per year)
00:00,60
00:15,60
00:30,60
00:45,60
01:00,60
01:15,60
01:30,60
01:45,60
02:00,60
02:15,60
02:30,60
... (96 lines in total: 00:00-17:45 and 22:00-23:45 = 60 Wh, 18:00-21:45 = 250 Wh) ...
23:15,60
23:30,60
23:45,60
```

# Section D - Task programs (Tasks 2-7)

## task2_types.c - Task 2

```
/* Task 2 - data types: sizes, ranges, float drift, 16-bit rollover, double vs paise */
#include <stdio.h>
#include <limits.h>
#include <float.h>
#include <stdint.h>
#include <time.h>

int main(void)
{
    printf("=== sizeof (bytes) on this machine ===\n");
    printf("char=%zu short=%zu int=%zu long=%zu long long=%zu float=%zu double=%zu\n",
           sizeof(char), sizeof(short), sizeof(int), sizeof(long), sizeof(long long),
           sizeof(float), sizeof(double));
    printf("unsigned short=%zu int32_t=%zu int64_t=%zu\n", sizeof(unsigned short), sizeof(int32_t), sizeof(int64_t));

    printf("\n=== Ranges from limits.h / float.h ===\n");
    printf("USHRT_MAX = %u\n", (unsigned)USHRT_MAX);
    printf("INT_MIN   = %d   INT_MAX = %d\n", INT_MIN, INT_MAX);
    printf("UINT_MAX  = %u\n", UINT_MAX);
    printf("LONG_MIN  = %ld   LONG_MAX = %ld\n", LONG_MIN, LONG_MAX);
    printf("LLONG_MAX = %lld\n", LLONG_MAX);
    printf("FLT_MAX = %e  FLT_DIG = %d  FLT_EPSILON = %e\n", FLT_MAX, FLT_DIG, FLT_EPSILON);
    printf("DBL_MAX = %e  DBL_DIG = %d  DBL_EPSILON = %e\n", DBL_MAX, DBL_DIG, DBL_EPSILON);

    printf("\n=== Float precision: adding Rs 1234.57 for 4000 bills ===\n");
    {
        float f = 0.0f; double d = 0.0; long paise = 0;
        for (int i = 0; i < 4000; ++i) { f += 1234.57f; d += 1234.57; paise += 123457L; }
        printf("expected exact total      : Rs 4938280.00\n");
        printf("float  sum (rupees)       : %.4f   error = %+.4f\n", f, f - 4938280.0f);
        printf("double sum (rupees)       : %.6f   error = %+.8f\n", d, d - 4938280.0);
        printf("integer paise sum         : %ld paise = Rs %ld.%02ld   error = 0\n", paise, paise / 100, paise % 100);
        printf("float: 16777216.0f + 1.0f == 16777216.0f ? %s\n", (16777216.0f + 1.0f == 16777216.0f) ? "yes (1 is lost!)" : "no");
        printf("0.1 + 0.2 == 0.3 (double) ? %s\n", (0.1 + 0.2 == 0.3) ? "yes" : "no");
    }

    printf("\n=== 16-bit unsigned meter register rollover ===\n");
    {
        unsigned short r = 65535;
        printf("r = 65535;  r = r + 1;  -> r = ");
        r = r + 1;
        printf("%u   (wraps modulo 65536)\n", r);

        unsigned short prev = 65530, curr = 4;          /* meter went 65530 -> 65535 -> 0 -> 4 */
        unsigned short units16 = (unsigned short)(curr - prev);
        long units_if = (curr >= prev) ? (long)curr - prev : 65536L - prev + curr;
        printf("prev=%u curr=%u : naive curr-prev (as int) = %d\n", prev, curr, curr - prev);
        printf("                : (unsigned short)(curr-prev) = %u  (modular arithmetic gives correct units)\n", units16);
        printf("                : 65536 - prev + curr         = %ld\n", units_if);
        printf("Total energy one 16-bit register can show: %ld units before wrapping\n", 65535L + 1);
    }

    printf("\n=== double rupees vs integer paise: 20,000,000 additions ===\n");
    {
        volatile double dsum = 0.0; volatile long psum = 0;
        clock_t t0 = clock();
        for (long i = 0; i < 20000000L; ++i) dsum += 0.01;
        clock_t t1 = clock();
        for (long i = 0; i < 20000000L; ++i) psum += 1;
        clock_t t2 = clock();
        printf("double: sum of 20,000,000 x 0.01 = %.6f (exact answer 200000.000000)   time %.3f s\n",
               dsum, (double)(t1 - t0) / CLOCKS_PER_SEC);
        printf("long  : sum of 20,000,000 x 1 paise = %ld paise = Rs %ld.%02ld exactly   time %.3f s\n",
               psum, psum / 100, psum % 100, (double)(t2 - t1) / CLOCKS_PER_SEC);
        printf("Space: double = %zu bytes, long = %zu bytes per amount\n", sizeof(double), sizeof(long));
    }
    return 0;
}
```

## task3_const.c - Task 3

```
/* Task 3 - constants: #define vs const vs enum; billing with macros; RATE2_PAISE change */
#include <stdio.h>
#include "tariff.h"

/* const-variable versions of the same tariff (for comparison only) */
static const long K_RATE1 = 150L, K_RATE2 = 200L, K_RATE3 = 300L;
/* enum version of the slab limits */
enum SlabLimit { E_SLAB1 = 100, E_SLAB2 = 200 };

static long bill_using_macros(long units)
{
    if (units <= SLAB1_LIMIT) return units * RATE1_PAISE + FIXED_CHARGE_PAISE[CAT_DOMESTIC];
    if (units <= SLAB2_LIMIT) return SLAB1_LIMIT * RATE1_PAISE + (units - SLAB1_LIMIT) * RATE2_PAISE + FIXED_CHARGE_PAISE[CAT_DOMESTIC];
    return SLAB1_LIMIT * RATE1_PAISE + (SLAB2_LIMIT - SLAB1_LIMIT) * RATE2_PAISE + (units - SLAB2_LIMIT) * RATE3_PAISE + FIXED_CHARGE_PAIS\
    E[CAT_DOMESTIC];
}

int main(void)
{
    static const long sample[] = { 0, 50, 100, 150, 200, 250, 300 };
```

```
    printf("Tariff in force: RATE1=%ld RATE2=%ld RATE3=%ld paise/unit, fixed(domestic)=%ld paise\n",
           (long)RATE1_PAISE, (long)RATE2_PAISE, (long)RATE3_PAISE, FIXED_CHARGE_PAISE[CAT_DOMESTIC]);
    printf("%-8s %-12s %-10s\n", "Units", "Total paise", "Total Rs");
    for (size_t i = 0; i < sizeof sample / sizeof sample[0]; ++i) {
        long p = bill_using_macros(sample[i]);
        printf("%-8ld %-12ld %ld.%02ld\n", sample[i], p, p / 100, p % 100);
    }
    printf("\n-- macro / const / enum comparison --\n");
    printf("RATE2 via #define = %ld, via const = %ld (same value: %s)  <- const needs its own manual edit\n", (long)RATE2_PAISE, K_RATE2, \
    (long)RATE2_PAISE == K_RATE2 ? "yes" : "NO - differs");
    printf("const object has an address & occupies memory: &K_RATE1 = %p, sizeof = %zu\n", (const void *)&K_RATE1, sizeof K_RATE1);
    printf("#define has no address/size of its own (it is replaced by the literal %ldL before compilation)\n", (long)RATE1_PAISE);
    printf("enum value E_SLAB2 = %d, sizeof(enum SlabLimit) = %zu\n", E_SLAB2, sizeof(enum SlabLimit));
    (void)K_RATE3; (void)E_SLAB1;
    switch (100) { case E_SLAB1: puts("enum/#define usable as case labels (compile-time constants)"); break; default: break; }
    /* case K_RATE1: would be a compile ERROR in C - const int is not a constant expression */
    return 0;
}
```

### task4_macros.c - Task 4

```
/* Task 4 - macro safety and macro-vs-function */
#include <stdio.h>
#include <time.h>
#include "tariff.h"

#define BAD_SQUARE(x) x * x            /* WRONG: no parentheses */
#define BAD_MAX(a, b) a > b ? a : b    /* WRONG: no parentheses */
static inline long max_inline(long a, long b) { return a > b ? a : b; }
__attribute__((noinline)) long sq_func(long x) { return x * x; }
static inline long sq_inline(long x) { return x * x; }

int main(void)
{
    printf("RUPEES_TO_PAISE(12.34) = %ld paise,  RUPEES_TO_PAISE(99.99) = %ld paise\n", RUPEES_TO_PAISE(12.34), RUPEES_TO_PAISE(99.99));
    printf("MAX(150, 230) = %d\n", MAX(150, 230));

    printf("\n-- SQUARE without parentheses --\n");
    printf("BAD_SQUARE(2+3)  expands to 2+3 * 2+3 = %d   (expected 25)\n", BAD_SQUARE(2 + 3));
    printf("SQUARE(2+3)      expands to ((2+3)*(2+3)) = %d\n", SQUARE(2 + 3));
    printf("BAD_MAX result inside expression: 10 + BAD_MAX(1,2) = %d (expected 12)\n", 10 + BAD_MAX(1, 2));

    printf("\n-- MAX(a,b) evaluates an argument twice --\n");
    {
        int i = 5, j = 3;
        int m = MAX(i++, j++);
        printf("macro   : MAX(i++, j++) -> m=%d, i=%d, j=%d  (i incremented TWICE)\n", m, i, j);
        i = 5; j = 3;
        long f = max_inline(i++, j++);
        printf("function: max_inline(i++, j++) -> m=%ld, i=%d, j=%d  (each argument evaluated once)\n", f, i, j);
    }

    printf("\n-- Speed: 300,000,000 iterations (volatile sink) --\n");
    {
        volatile long sink = 0; clock_t t;
        t = clock(); for (long i = 0; i < 300000000L; ++i) sink += SQUARE(i & 1023);
        printf("macro SQUARE        : %.3f s\n", (double)(clock() - t) / CLOCKS_PER_SEC);
        t = clock(); for (long i = 0; i < 300000000L; ++i) sink += sq_inline(i & 1023);
        printf("static inline       : %.3f s\n", (double)(clock() - t) / CLOCKS_PER_SEC);
        t = clock(); for (long i = 0; i < 300000000L; ++i) sink += sq_func(i & 1023);
        printf("noinline function   : %.3f s\n", (double)(clock() - t) / CLOCKS_PER_SEC);
    }
    return 0;
}
```

### macro_expand.c - Task 4 - gcc -E demo

```
#include "tariff.h"
long demo(long units) { return SQUARE(units + 1) + MAX(units, 100) + RUPEES_TO_PAISE(2.5); }
long rate(void) { return RATE2_PAISE; }
```

### task5_debug.c - Task 5

```
/* Task 5 - include guards, conditional debugging (#ifdef DEBUG), runtime vs compile-time debug */
#include <stdio.h>
#include <string.h>
#include "tariff.h"
#include "tariff.h"        /* included twice on purpose: include guard makes this harmless */

int main(int argc, char **argv)
{
    int runtime_debug = (argc > 1 && strcmp(argv[1], "--debug") == 0);
    Customer c = { 3001, CAT_DOMESTIC, 1000, 1180, 0 };
    Bill b;
    int st = generate_bill(&c, &b);          /* DBG() inside prints only if compiled with -DDEBUG */
    DBG("main: generate_bill returned %d", st);
    if (runtime_debug) fprintf(stderr, "[RUNTIME-DEBUG] status=%d units=%ld total=%ld\n", st, b.units, b.total_paise);
    printf("Bill for HH %ld: units=%ld total=", c.household_no, b.units);
    print_money(b.total_paise); printf("\n");
    return 0;
}
```

### noguard.h - Task 5 - header without guard

```
/* header WITHOUT include guard - demonstrates redefinition error */
struct Point { int x, y; };
```

### noguard_demo.c - Task 5

```
#include "noguard.h"
#include "noguard.h"
int main(void) { struct Point p = {1, 2}; return p.x - 1; }
```

### task6_io.c - Task 6

```
/* Task 6 - formatted input/output with scanf / printf */
#include <stdio.h>
int main(void)
{
    int hh; unsigned int u; long l; long long ll; float f; double d; char c; char s[32];
    int n = scanf("%d %u %ld %lld %f %lf %c %31s", &hh, &u, &l, &ll, &f, &d, &c, s);
    if (n != 8) { printf("INVALID INPUT: scanf converted only %d of 8 fields -> rejecting\n", n); return 1; }
    printf("%%d  int           : %d\n", hh);
    printf("%%u  unsigned int  : %u\n", u);
    printf("%%ld long          : %ld\n", l);
    printf("%%lld long long    : %lld\n", ll);
    printf("%%f  float         : %f   (%%.2f -> %.2f)\n", f, f);
    printf("%%lf double (scanf): %lf  (printf uses %%f/%%lf for double)\n", d);
    printf("%%c  char          : %c\n", c);
    printf("%%s  string        : %s\n", s);
    long paise = (long)u;   /* amount stored in paise */
    printf("Amount %ld paise  ->  Rs %ld and %02ld paise  (Rs %ld.%02ld)\n", paise, paise / 100, paise % 100, paise / 100, paise % 100);
    return 0;
}
```

### task6_wrong.c - Task 6 - wrong specifiers

```
/* Task 6 - incorrect format specifiers (undefined behaviour; compile with -Wall -Wformat) */
#include <stdio.h>
int main(void)
{
    long long big = 5000000000LL;
    printf("printf(\"%%d\", 3.14)        -> %d\n", 3.14);
    printf("printf(\"%%f\", 42)          -> %f\n", 42);
    printf("printf(\"%%d\", 5000000000LL)-> %d   (high bits lost)\n", big);
    printf("printf(\"%%u\", -1)          -> %u\n", -1);
    printf("printf(\"%%c\", 321)         -> %c   (321 mod 256 = 65)\n", 321);
    return 0;
}
```

### task6_compare.c - Task 6 - scanf vs strtol

```
/* Task 6 - scanf vs fgets()+sscanf() vs fgets()+strtol() */
#include <stdio.h>
#include <stdlib.h>
#include <errno.h>
int main(void)
{
    char line[64];
    printf("%-14s | %-26s | %-34s\n", "input line", "sscanf(\"%d\") (ret,val)", "strtol (value, leftover, errno)");
    while (fgets(line, sizeof line, stdin)) {
        int v = 0, r = sscanf(line, "%d", &v);
        char *end; errno = 0; long t = strtol(line, &end, 10);
        int junk = 0; for (char *p = end; *p; ++p) if (*p != ' ' && *p != '\n') junk = 1;
        char shown[40]; int k = 0; for (char *p = line; *p && *p != '\n' && k < 38; ++p) shown[k++] = *p; shown[k] = 0;
        printf("%-14s | ret=%d val=%-11d | val=%-11ld %s errno=%d\n", shown, r, v, t,
               (end == line) ? "NO-DIGITS" : junk ? "TRAILING-JUNK" : "clean", errno);
    }
    return 0;
}
```

### task7_conv.c - Task 7

```
/* Task 7 - type conversion and precision */
#include <stdio.h>
#include <math.h>
int main(void)
{
    printf("== 1. Integer promotion ==\n");
    unsigned short a = 60000, b = 60000;
    printf("a + b = %d (both promoted to int, so NO 16-bit wrap); (unsigned short)(a+b) = %u\n", a + b, (unsigned)(unsigned short)(a + b))\
    ;
    char c1 = 100, c2 = 100;
    printf("char 100 + char 100 = %d (promoted to int; not -56)\n", c1 + c2);

    printf("\n== 2. Usual arithmetic conversions (signed vs unsigned) ==\n");
    int neg = -1; unsigned int one = 1u;
    printf("(-1 < 1u) is %d   <- UNEXPECTED: -1 is converted to 4294967295u\n", neg < one);

    printf("\n== 3. Integer division errors ==\n");
    printf("7 / 2 = %d,  7 / 2.0 = %.1f,  1 / 3 * 3.0 = %.1f (UNEXPECTED: 0)\n", 7 / 2, 7 / 2.0, 1 / 3 * 3.0);
    long units = 250, total = 52500;
    printf("avg paise/unit: total/units = %ld (integer, truncated) vs (double) = %.2f\n", total / units, (double)total / units);
    printf("(units/100)*100 = %ld  (not 250) <- slab rounding bug\n", (units / 100) * 100);

    printf("\n== 4. Explicit casts and narrowing ==\n");
    long big = 5000000000L;
    printf("(int)5000000000L = %d  (truncated; sizeof(long)=%zu, sizeof(int)=%zu)\n", (int)big, sizeof(long), sizeof(int));
    printf("(unsigned char)300 = %d,  (int)3.99 = %d,  (int)-3.99 = %d\n", (unsigned char)300, (int)3.99, (int)-3.99);

    printf("\n== 5. Rounding fractional charge to paise: 33 units x Rs 1.505 ==\n");
    double rupees = 33 * 1.505;
    printf("rupees (double)        = %.17f\n", rupees);
    printf("(long)(rupees*100)     = %ld   <- truncation (wrong)\n", (long)(rupees * 100));
```

```
    printf("lround(rupees*100)     = %ld   <- right here only because rupees*100 rounds to 4966.5; not guaranteed\n", lround(rupees * 100)\
    );
    printf("integer: (33*1505+5)/10= %ld   <- exact (rate kept as 1505 milli-rupees)\n", (33L * 1505L + 5L) / 10L);

    printf("\n== 6. Other unexpected expression results ==\n");
    printf("0.1 + 0.2 == 0.3 ? %d\n", 0.1 + 0.2 == 0.3);
    float fs = 0; for (int i = 0; i < 10; ++i) fs += 0.1f;
    printf("float: ten times 0.1f = %.8f, == 1.0f ? %d\n", fs, fs == 1.0f);
    printf("sizeof(char) > -1 ? %d  (sizeof is size_t, unsigned: -1 becomes huge -> 0)\n", sizeof(char) > -1);
    printf("5 / 2 * 2.0 = %.1f  but 5 * 2.0 / 2 = %.1f\n", 5 / 2 * 2.0, 5 * 2.0 / 2);

    printf("\n== 7. Accuracy: float vs integer paise (1,000,000 x Rs 0.07) ==\n");
    float fsum = 0; double dsum = 0; long psum = 0;
    for (int i = 0; i < 1000000; ++i) { fsum += 0.07f; dsum += 0.07; psum += 7; }
    printf("exact = Rs 70000.00 | float = %.4f (err %+.4f) | double = %.6f (err %+.8f) | paise = %ld.%02ld\n",
           fsum, fsum - 70000.0, dsum, dsum - 70000.0, psum / 100, psum % 100);
    printf("Space: float 4 B, double 8 B, long 8 B (int32 4 B).  Time: integer add is 1 cycle; FP add 3-5 cycles latency.\n");
    return 0;
}
```

# Section E - Task 8: compilation process

### err_compile.c - compile-time error

```
#include <stdio.h>
int main(void)
{
    int units = 100          /* missing semicolon -> COMPILE-TIME (syntax) error */
    printf("%d\n", units);
    return 0;
}
```

### err_link.c - link-time error

```
#include <stdio.h>
long energy_charge_paise(long units);      /* declared, but billing.c is NOT linked */
int main(void) { printf("%ld\n", energy_charge_paise(150)); return 0; }
```

### err_runtime.c - runtime error

```
#include <stdio.h>
int main(void)
{
    volatile long units = 0, total = 5000;
    printf("average paise per unit = %ld\n", total / units);   /* compiles & links fine; crashes when run */
    return 0;
}
```

### warn.c - warnings

```
#include <stdio.h>
int count_over(int n, const int *units, int limit)   /* 'limit' unused */
{
    int c;                                           /* used uninitialised */
    size_t i;
    int unused_var = 5;                              /* unused variable    */
    for (i = 0; i < n; ++i)                          /* signed/unsigned compare */
        if (units[i] > 100) c++;
    return c;
}
int main(void) { int u[3] = {50, 150, 250}; printf("%d\n", count_over(3, u, 200)); return 0; }
```

### warn_fixed.c - warnings resolved

```
#include <stdio.h>
int count_over(int n, const int *units, int limit)
{
    int c = 0;                                       /* initialised */
    for (int i = 0; i < n; ++i)                      /* same signedness */
        if (units[i] > limit) c++;                   /* parameter now used */
    return c;
}
int main(void) { int u[3] = {50, 150, 250}; printf("%d\n", count_over(3, u, 100)); return 0; }
```

### run_task8.sh - script that runs all stages

```
#!/bin/bash
cd "$(dirname "$0")"; mkdir -p stages outputs; O=outputs/task8.txt
{
echo '===== STAGE 1: PREPROCESSING  (gcc -E) ====='
echo '$ gcc -E main.c -o stages/main.i'; gcc -E main.c -o stages/main.i
echo '$ wc -l main.c stages/main.i'; wc -l main.c stages/main.i
echo '$ grep -c "^#define" stages/main.i   (macros are gone after preprocessing)'; grep -c '^#define' stages/main.i
echo '$ grep -n "RATE\|DBG\|TARIFF_H" stages/main.i | head -3   (no macro names remain)'; grep -n 'RATE\|DBG\|TARIFF_H' stages/main.i | he\
    ad -3; echo '(no output = none left)'
echo '$ tail -n 6 stages/main.i   (main() body: #include expanded, comments removed)'; tail -n 6 stages/main.i | cut -c1-110
echo
echo '===== STAGE 2: COMPILATION TO ASSEMBLY (gcc -S) ====='
echo '$ gcc -S main.c -o stages/main.s'; gcc -S main.c -o stages/main.s
echo '$ head -n 12 stages/main.s'; head -n 12 stages/main.s
echo
echo '===== STAGE 3: ASSEMBLY TO OBJECT (gcc -c) ====='
echo '$ gcc -c main.c -o stages/main.o ; gcc -c billing.c -o stages/billing.o'; gcc -c main.c -o stages/main.o; gcc -c billing.c -o stages\
    /billing.o
echo '$ file stages/main.o'; file stages/main.o
echo '$ nm stages/main.o | grep " U "   (U = undefined, resolved only at link time)'; nm stages/main.o | grep ' U '
echo '$ nm stages/billing.o | grep " T "   (T = functions defined here)'; nm stages/billing.o | grep ' T '
echo
echo '===== STAGE 4: LINKING ====='
echo '$ gcc stages/main.o stages/billing.o -o stages/billing_linked'; gcc stages/main.o stages/billing.o -o stages/billing_linked && echo \
    'linked OK'
echo '$ ./stages/billing_linked < inputs/five_bills.txt | tail -3'; ./stages/billing_linked < inputs/five_bills.txt | tail -3
echo
echo '===== COMPILE-TIME vs LINK-TIME vs RUNTIME ERRORS ====='
echo '$ gcc -o err_compile err_compile.c'; gcc -o err_compile err_compile.c 2>&1 | head -6
echo '$ gcc -o err_link err_link.c'; gcc -o err_link err_link.c 2>&1 | grep -v 'warning' | head -4
echo '$ gcc -o err_runtime err_runtime.c && ./err_runtime'; gcc -o err_runtime err_runtime.c && ./err_runtime; echo "exit status = $? (136\
     = 128 + SIGFPE)"
echo
echo '===== WARNINGS: gcc -Wall -Wextra -O2 warn.c ====='
echo '$ gcc -Wall -Wextra -O2 -c warn.c'; gcc -Wall -Wextra -O2 -c warn.c -o /dev/null 2>&1 | grep -E 'warning'  | sed 's/^warn.c://'
echo '--- after fixing (warn_fixed.c) ---'
echo '$ gcc -Wall -Wextra -O2 -o warn_fixed warn_fixed.c'; gcc -Wall -Wextra -O2 -o warn_fixed warn_fixed.c && echo '(no warnings)'; ./war\
    n_fixed
echo
echo '===== SINGLE-FILE vs SEPARATE COMPILATION ====='
```

```
TIMEFORMAT='   real %3Rs'
echo '$ time gcc -o billing_single main.c billing.c   (everything recompiled)'; time gcc -o stages/billing_single main.c billing.c
echo '$ gcc -c billing.c ; gcc -c main.c ; gcc main.o billing.o   (initial separate build)'; gcc -c billing.c -o stages/billing.o; gcc -c \
    main.c -o stages/main.o; gcc stages/main.o stages/billing.o -o stages/billing_sep
echo '(edit only main.c ...) $ time { gcc -c main.c ; gcc main.o billing.o }   (billing.o reused)'; time { gcc -c main.c -o stages/main.o;\
     gcc stages/main.o stages/billing.o -o stages/billing_sep; }
} > $O 2>&1
```

# Section F - Tests, scale test and smart meter (Tasks 9, 10, 12)

### test_slabs.c - Task 9

```
/* Task 9 - compare fixed slab conditions (if-else) with array+loop; boundary assertions; timing */
#include <stdio.h>
#include <time.h>
#include "tariff.h"
int main(void)
{
    int mismatches = 0;
    for (long u = 0; u <= 1000; ++u)
        if (energy_charge_paise(u) != energy_charge_paise_array(u)) { ++mismatches; printf("MISMATCH at %ld\n", u); }
    printf("Compared if-else vs array/loop for units 0..1000 -> mismatches = %d\n", mismatches);
    long chk[][2] = { {0, 0}, {100, 15000}, {101, 15200}, {200, 35000}, {201, 35300} };
    for (size_t i = 0; i < sizeof chk / sizeof chk[0]; ++i)
        printf("units=%-4ld expected energy=%-6ld if-else=%-6ld array=%-6ld %s\n", chk[i][0], chk[i][1],
               energy_charge_paise(chk[i][0]), energy_charge_paise_array(chk[i][0]),
               (energy_charge_paise(chk[i][0]) == chk[i][1] && energy_charge_paise_array(chk[i][0]) == chk[i][1]) ? "PASS" : "FAIL");
    volatile long sink = 0; clock_t t = clock();
    for (long r = 0; r < 20000; ++r) for (long u = 0; u < 1000; ++u) sink += energy_charge_paise(u);
    double t1 = (double)(clock() - t) / CLOCKS_PER_SEC; t = clock();
    for (long r = 0; r < 20000; ++r) for (long u = 0; u < 1000; ++u) sink += energy_charge_paise_array(u);
    double t2 = (double)(clock() - t) / CLOCKS_PER_SEC;
    printf("Time for 20,000,000 bill calculations: if-else = %.3f s, array+loop = %.3f s\n", t1, t2);
    return 0;
}
```

### task10_scale.c - Task 10

```
/* Task 10 - float vs integer-paise totals; int/long/long long overflow at district scale */
#include <stdio.h>
#include <stdint.h>
#include <limits.h>
#include <time.h>
#include "tariff.h"

static long bill_for(long i)            /* deterministic pseudo-consumption: 0..449 units */
{
    long units = (i * 37L) % 450L;
    return energy_charge_paise(units) + FIXED_CHARGE_PAISE[i % 3];
}

int main(void)
{
    static const long N[] = { 4000L, 60000L, 800000L };
    printf("%-9s %-18s %-20s %-18s %-14s %-14s\n", "Bills", "exact (int64 paise)", "float rupees", "double rupees", "int32 total", "long t\
    otal");
    for (int k = 0; k < 3; ++k) {
        float fsum = 0.0f; double dsum = 0.0; long long exact = 0; int32_t i32 = 0; long lng = 0; int ovf32 = 0;
        for (long i = 0; i < N[k]; ++i) {
            long p = bill_for(i);
            fsum += (float)p / 100.0f; dsum += (double)p / 100.0; exact += p; lng += p;
            if (__builtin_add_overflow(i32, (int32_t)p, &i32)) ovf32 = 1;
        }
        printf("%-9ld Rs %-15lld %-20.2f %-18.4f %-14s %ld\n", N[k], exact / 100, fsum, dsum, ovf32 ? "OVERFLOW" : "ok", lng);
        printf("          exact = %lld paise | float error = %+.2f Rs | double error = %+.6f Rs | int32 wrapped value = %d\n",
               exact, fsum - (double)exact / 100.0, dsum - (double)exact / 100.0, i32);
    }

    printf("\n-- long long vs double for district totals (exactness limit) --\n");
    long long big = 9007199254740993LL;             /* 2^53 + 1 */
    double dbig = (double)big;
    printf("2^53+1 as long long = %lld ; stored in double = %.0f  (double cannot hold it exactly)\n", big, dbig);
    printf("Largest district total (800,000 bills x 12 months x Rs 50,000 max) = %lld paise (< 2^53 = 9007199254740992, so both fit)\n", 8\
    00000LL * 12 * 5000000LL);
    printf("Max values: INT_MAX=%d (Rs %d.%02d), LONG_MAX=%ld, LLONG_MAX=%lld\n", INT_MAX, INT_MAX / 100, INT_MAX % 100, LONG_MAX, LLONG_M\
    AX);

    printf("\n-- Actual vs minimum guaranteed by C standard --\n");
    printf("%-12s %-12s %-14s %-s\n", "type", "this system", "C standard min", "actual range");
    printf("%-12s %-12zu %-14s %d .. %d\n", "int", sizeof(int), "2 B (16 bit)", INT_MIN, INT_MAX);
    printf("%-12s %-12zu %-14s %ld .. %ld\n", "long", sizeof(long), "4 B (32 bit)", LONG_MIN, LONG_MAX);
    printf("%-12s %-12zu %-14s %lld .. %lld\n", "long long", sizeof(long long), "8 B (64 bit)", LLONG_MIN, LLONG_MAX);
    printf("%-12s %-12zu %-14s 0 .. %u\n", "unsigned short", sizeof(unsigned short), "2 B (16 bit)", (unsigned)USHRT_MAX);
    printf("Standard guarantees only: int >= -32767..32767, long >= +-2147483647, long long >= +-9223372036854775807\n");

    printf("\n-- Time: 50,000,000 additions --\n");
    volatile double ds = 0; volatile long long ls = 0; clock_t t = clock();
    for (long i = 0; i < 50000000L; ++i) ds += 12.34;
    double td = (double)(clock() - t) / CLOCKS_PER_SEC; t = clock();
    for (long i = 0; i < 50000000L; ++i) ls += 1234;
    double tl = (double)(clock() - t) / CLOCKS_PER_SEC;
    printf("double = %.3f s   long long = %.3f s   (both 8 bytes of space)\n", td, tl);
    return 0;
}
```

### smart_meter.c - Task 12

```
/* Task 12 - smart-meter billing: Wh readings every 15 min, 64-bit totals, runtime tariff, peak pricing */
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <stdint.h>
#include <inttypes.h>
```

```
typedef struct {
    int64_t slab1_wh, slab2_wh;            /* slab limits in Wh             */
    int64_t rate1, rate2, rate3;           /* paise per kWh                 */
    int     peak_start, peak_end, peak_pct; /* hours [start,end), surcharge % */
    int64_t fixed_paise;
} Tariff;

static int load_tariff(const char *path, Tariff *t)
{
    FILE *fp = fopen(path, "r"); char line[128], key[64]; long long v; int found = 0;
    if (!fp) { perror("tariff file"); return -1; }
    while (fgets(line, sizeof line, fp)) {
        if (line[0] == '#' || sscanf(line, "%63s %lld", key, &v) != 2) continue;
        if      (!strcmp(key, "SLAB1_LIMIT_KWH"))        { t->slab1_wh = v * 1000; ++found; }
        else if (!strcmp(key, "SLAB2_LIMIT_KWH"))        { t->slab2_wh = v * 1000; ++found; }
        else if (!strcmp(key, "RATE1_PAISE_PER_KWH"))    { t->rate1 = v; ++found; }
        else if (!strcmp(key, "RATE2_PAISE_PER_KWH"))    { t->rate2 = v; ++found; }
        else if (!strcmp(key, "RATE3_PAISE_PER_KWH"))    { t->rate3 = v; ++found; }
        else if (!strcmp(key, "PEAK_START_HOUR"))        { t->peak_start = (int)v; ++found; }
        else if (!strcmp(key, "PEAK_END_HOUR"))          { t->peak_end = (int)v; ++found; }
        else if (!strcmp(key, "PEAK_SURCHARGE_PERCENT")) { t->peak_pct = (int)v; ++found; }
        else if (!strcmp(key, "FIXED_CHARGE_PAISE"))     { t->fixed_paise = v; ++found; }
    }
    fclose(fp);
    return found == 9 ? 0 : -1;
}

/* cost of `wh` watt-hours when `cum` Wh were already consumed, in (paise x Wh) units */
static int64_t slab_cost_pwh(const Tariff *t, int64_t cum, int64_t wh)
{
    int64_t cost = 0;
    const int64_t upto[3] = { t->slab1_wh, t->slab2_wh, INT64_MAX };
    const int64_t rate[3] = { t->rate1, t->rate2, t->rate3 };
    for (int i = 0; i < 3 && wh > 0; ++i) {
        if (cum >= upto[i]) continue;
        int64_t room = upto[i] - cum, take = wh < room ? wh : room;
        cost += take * rate[i]; cum += take; wh -= take;
    }
    return cost;
}

int main(int argc, char **argv)
{
    const char *cfg = argc > 1 ? argv[1] : "tariff.cfg";
    Tariff t;
    if (load_tariff(cfg, &t) != 0) { fprintf(stderr, "bad tariff file %s\n", cfg); return 1; }
    printf("Loaded tariff from %s: slabs %" PRId64 "/%" PRId64 " Wh, rates %" PRId64 "/%" PRId64 "/%" PRId64
           " paise/kWh, peak %02d:00-%02d:00 +%d%%, fixed %" PRId64 " paise\n",
           cfg, t.slab1_wh, t.slab2_wh, t.rate1, t.rate2, t.rate3, t.peak_start, t.peak_end, t.peak_pct, t.fixed_paise);

    /* read one day of readings */
    int hour[96], n = 0; int64_t wh[96]; char line[64];
    FILE *fp = fopen("inputs/readings_day.csv", "r");
    if (!fp) { perror("readings"); return 1; }
    while (fgets(line, sizeof line, fp) && n < 96) {
        int h, m; long long w;
        if (line[0] == '#' || sscanf(line, "%d:%d,%lld", &h, &m, &w) != 3) continue;
        hour[n] = h; wh[n] = w; ++n;
    }
    fclose(fp);
    printf("Readings per day = %d, per year = %d (365 x %d)\n", n, 365 * n, n);

    /* bill one 30-day month, interval by interval (chronological slab allocation) */
    int64_t cum = 0, peak_wh = 0, normal_pwh = 0, peak_pwh = 0;
    for (int day = 0; day < 30; ++day)
        for (int i = 0; i < n; ++i) {
            int is_peak = hour[i] >= t.peak_start && hour[i] < t.peak_end;
            int64_t c = slab_cost_pwh(&t, cum, wh[i]);
            if (is_peak) { peak_pwh += c; peak_wh += wh[i]; } else normal_pwh += c;
            cum += wh[i];
        }
    int64_t scaled = normal_pwh * 100 + peak_pwh * (100 + t.peak_pct);       /* paise x Wh x 100 */
    int64_t energy = (scaled + 50000) / 100000;                               /* /1000 (Wh->kWh), /100 (%), rounded */
    int64_t total  = energy + t.fixed_paise;
    printf("\n-- Monthly bill (30 days = %d readings) --\n", 30 * n);
    printf("Total consumption : %" PRId64 " Wh (%.3f kWh)   [16-bit register would wrap %" PRId64 " times]\n", cum, cum / 1000.0, cum / 65\
    536);
    printf("Peak consumption  : %" PRId64 " Wh   Off-peak: %" PRId64 " Wh\n", peak_wh, cum - peak_wh);
    printf("Energy charge     : %" PRId64 " paise   Fixed: %" PRId64 " paise   TOTAL: %" PRId64 " paise = Rs %" PRId64 ".%02" PRId64 "\n",
           energy, t.fixed_paise, total, total / 100, total % 100);
    /* same month WITHOUT peak surcharge, for evidence */
    int64_t flat = (normal_pwh + peak_pwh + 500) / 1000 + t.fixed_paise;
    printf("Same month without peak rule: %" PRId64 " paise -> peak rule adds %" PRId64 " paise\n", flat, total - flat);

    /* Impact analysis: largest possible values (assumed maximum load 10 kW per household) */
    const int64_t MAX_W = 10000, READINGS = 35040, HOUSEHOLDS = 800000, MAXRATE = 300;
    int64_t max_wh_interval = MAX_W / 4;                        /* 15 min = 1/4 hour */
    int64_t max_year_wh     = max_wh_interval * READINGS;
    int64_t max_district_wh = max_year_wh * HOUSEHOLDS;
    int64_t max_pwh_house   = max_year_wh * MAXRATE * (100 + t.peak_pct);   /* paise x Wh x 100 */
    printf("\n-- Largest possible values (10 kW max load, %" PRId64 " households) --\n", HOUSEHOLDS);
    printf("Max Wh per 15-min reading      : %" PRId64 "   (16-bit max 65535 -> fits)\n", max_wh_interval);
    printf("Max Wh per household per year  : %" PRId64 "   (uint32 max 4294967295 -> %s)\n", max_year_wh, max_year_wh > 4294967295LL ? "OV\
    ERFLOW" : "fits uint32, NOT 16-bit");
    printf("Max Wh for whole district/year : %" PRId64 "   (uint32 max 4294967295 -> %s)\n", max_district_wh, max_district_wh > 4294967295\
    LL ? "needs 64-bit" : "fits");
```

```
    printf("Max intermediate paise*Wh*100  : %" PRId64 "   (int32 max 2147483647 -> %s)\n", max_pwh_house, max_pwh_house > 2147483647LL ? \
    "needs 64-bit" : "fits");
    printf("INT64_MAX                      : %" PRId64 "\n", (int64_t)INT64_MAX);
    return 0;
}
```

# Section G - Execution output (evidence for every task)

Each block is the unedited terminal output saved by the build scripts (outputs/*.txt). Timing values vary slightly between runs.

### Task 2 - data types

```
$ gcc -Wall -Wextra -o task2_types task2_types.c
$ ./task2_types
=== sizeof (bytes) on this machine ===
char=1 short=2 int=4 long=8 long long=8 float=4 double=8
unsigned short=2 int32_t=4 int64_t=8

=== Ranges from limits.h / float.h ===
USHRT_MAX = 65535
INT_MIN   = -2147483648   INT_MAX = 2147483647
UINT_MAX  = 4294967295
LONG_MIN  = -9223372036854775808   LONG_MAX = 9223372036854775807
LLONG_MAX = 9223372036854775807
FLT_MAX = 3.402823e+38  FLT_DIG = 6  FLT_EPSILON = 1.192093e-07
DBL_MAX = 1.797693e+308  DBL_DIG = 15  DBL_EPSILON = 2.220446e-16

=== Float precision: adding Rs 1234.57 for 4000 bills ===
expected exact total      : Rs 4938280.00
float  sum (rupees)       : 4938160.0000   error = -120.0000
double sum (rupees)       : 4938280.000000   error = -0.00000007
integer paise sum         : 493828000 paise = Rs 4938280.00   error = 0
float: 16777216.0f + 1.0f == 16777216.0f ? yes (1 is lost!)
0.1 + 0.2 == 0.3 (double) ? no

=== 16-bit unsigned meter register rollover ===
r = 65535;  r = r + 1;  -> r = 0   (wraps modulo 65536)
prev=65530 curr=4 : naive curr-prev (as int) = -65526
                  : (unsigned short)(curr-prev) = 10  (modular arithmetic gives correct units)
                  : 65536 - prev + curr         = 10
Total energy one 16-bit register can show: 65536 units before wrapping

=== double rupees vs integer paise: 20,000,000 additions ===
double: sum of 20,000,000 x 0.01 = 200000.000034 (exact answer 200000.000000)   time 0.057 s
long  : sum of 20,000,000 x 1 paise = 20000000 paise = Rs 200000.00 exactly   time 0.011 s
Space: double = 8 bytes, long = 8 bytes per amount
```

### Task 3 - constants, RATE2_PAISE 200 and 230

```
--- Run 1: original RATE2_PAISE = 200 ---
$ gcc -Wall -Wextra -o task3 task3_const.c billing.c
$ ./task3
Tariff in force: RATE1=150 RATE2=200 RATE3=300 paise/unit, fixed(domestic)=5000 paise
Units    Total paise  Total Rs
0        5000         50.00
50       12500        125.00
100      20000        200.00
150      30000        300.00
200      40000        400.00
250      55000        550.00
300      70000        700.00

-- macro / const / enum comparison --
RATE2 via #define = 200, via const = 200 (same value: yes)  <- const needs its own manual edit
const object has an address & occupies memory: &K_RATE1 = 0x55dcca049020, sizeof = 8
#define has no address/size of its own (it is replaced by the literal 150L before compilation)
enum value E_SLAB2 = 200, sizeof(enum SlabLimit) = 4
enum/#define usable as case labels (compile-time constants)

--- Run 2: RATE2_PAISE changed to 230 (no source edit: -DRATE2_PAISE=230) ---
$ gcc -Wall -Wextra -DRATE2_PAISE=230 -o task3_r230 task3_const.c billing.c
$ ./task3_r230
Tariff in force: RATE1=150 RATE2=230 RATE3=300 paise/unit, fixed(domestic)=5000 paise
Units    Total paise  Total Rs
0        5000         50.00
50       12500        125.00
100      20000        200.00
150      31500        315.00
200      43000        430.00
250      58000        580.00
300      73000        730.00

-- macro / const / enum comparison --
RATE2 via #define = 230, via const = 200 (same value: NO - differs)  <- const needs its own manual edit
const object has an address & occupies memory: &K_RATE1 = 0x562475f5c020, sizeof = 8
#define has no address/size of its own (it is replaced by the literal 150L before compilation)
enum value E_SLAB2 = 200, sizeof(enum SlabLimit) = 4
enum/#define usable as case labels (compile-time constants)

--- Five bills re-run with RATE2_PAISE=230 ---
Line   HH    Category    Reading        Units       Energy      Fixed        Total
1      2001  Domestic     1000->1050        50         7500       5000        12500   (Rs 125.00)
2      2002  Domestic     1000->1150       150        26500       5000        31500   (Rs 315.00)
3      2003  Domestic     1000->1300       300        68000       5000        73000   (Rs 730.00)
4      2004  Commercial    500->620        120        19600      15000        34600   (Rs 346.00)
5      2005  Agriculture  2000->2250       250        53000       2500        55500   (Rs 555.00)

Valid bills: 5   Rejected: 0   Grand total: Rs 2071.00  (207100 paise)
```

### Task 4 - macro expansion (gcc -E)

```
$ cat macro_expand.c
#include "tariff.h"
long demo(long units) { return SQUARE(units + 1) + MAX(units, 100) + RUPEES_TO_PAISE(2.5); }
long rate(void) { return RATE2_PAISE; }

$ gcc -E macro_expand.c | tail -n 4
void print_money(long paise);
# 2 "macro_expand.c" 2
long demo(long units) { return ((units + 1) * (units + 1)) + (((units) > (100)) ? (units) : (100)) + ((long)((2.5) * 100.0 + 0.5)); }
long rate(void) { return 200L; }
```

### Task 4 - macro safety and timing

```
$ gcc -Wall -Wextra -O0 -o task4_macros task4_macros.c
$ ./task4_macros   (-O0)
RUPEES_TO_PAISE(12.34) = 1234 paise,  RUPEES_TO_PAISE(99.99) = 9999 paise
MAX(150, 230) = 230

-- SQUARE without parentheses --
BAD_SQUARE(2+3)  expands to 2+3 * 2+3 = 11   (expected 25)
SQUARE(2+3)      expands to ((2+3)*(2+3)) = 25
BAD_MAX result inside expression: 10 + BAD_MAX(1,2) = 1 (expected 12)

-- MAX(a,b) evaluates an argument twice --
macro    : MAX(i++, j++) -> m=6, i=7, j=4  (i incremented TWICE)
function: max_inline(i++, j++) -> m=5, i=6, j=4  (each argument evaluated once)

-- Speed: 300,000,000 iterations (volatile sink) --
macro SQUARE        : 0.358 s
static inline       : 0.396 s
noinline function   : 0.424 s

$ gcc -O2 -o task4_macros_O2 task4_macros.c ; ./task4_macros_O2   (-O2)
macro SQUARE        : 0.194 s
static inline       : 0.153 s
noinline function   : 0.441 s
```

### Task 5 - guards, DEBUG, include forms

```
$ gcc -Wall -Wextra -o t5_nodebug task5_debug.c billing.c
$ ./t5_nodebug
Bill for HH 3001: units=180 total=Rs 360.00

$ gcc -Wall -Wextra -DDEBUG -o t5_debug task5_debug.c billing.c
$ ./t5_debug
[DEBUG] billing.c:82: hh=3001 prev=1000 curr=1180 units=180 energy=31000 fixed=5000 total=36000
[DEBUG] task5_debug.c:13: main: generate_bill returned 0
Bill for HH 3001: units=180 total=Rs 360.00

$ ./t5_nodebug --debug   (runtime debugging via command-line flag)
[RUNTIME-DEBUG] status=0 units=180 total=36000
Bill for HH 3001: units=180 total=Rs 360.00

$ ls -l t5_nodebug t5_debug   (executable size, bytes)
16824 t5_debug
16824 t5_nodebug

$ size t5_nodebug t5_debug
   text       data        bss        dec        hex    filename
   4560        680         16       5256       1488    t5_nodebug
   4885        680         16       5581       15cd    t5_debug

--- include guard: header WITHOUT guard included twice ---
$ gcc noguard_demo.c
In file included from noguard_demo.c:2:
noguard.h:2:8: error: redefinition of 'struct Point'
    2 | struct Point { int x, y; };
      |        ^~~~~
In file included from noguard_demo.c:1:
noguard.h:2:8: note: originally defined here
    2 | struct Point { int x, y; };
      |        ^~~~~

--- #include <file> vs #include "file" ---
$ gcc -c angle.c            (angle brackets: system paths only)
angle.c:1:10: fatal error: tariff.h: No such file or directory
    1 | #include <tariff.h>
      |          ^~~~~~~~~~
compilation terminated.
$ gcc -I. -c angle.c        (-I. adds current dir to the <> search path)
compiled OK
$ gcc -c quote.c            (quotes: current dir first, then system paths)
compiled OK

--- system include search path (excerpt) ---
#include <...> search starts here:
 /usr/lib/gcc/x86_64-linux-gnu/13/include
 /usr/local/include
 /usr/include/x86_64-linux-gnu
 /usr/include
End of search list.
```

### Task 6 - input/output

```
$ gcc -Wall -Wextra -o task6_io task6_io.c
```

```
$ cat inputs/input_valid.txt
1001 12345 600000 8000000000 1.505 2.7182818 D Surya
$ ./task6_io < inputs/input_valid.txt
%d  int           : 1001
%u  unsigned int  : 12345
%ld long          : 600000
%lld long long    : 8000000000
%f  float         : 1.505000   (%.2f -> 1.50)
%lf double (scanf): 2.718282  (printf uses %f/%lf for double)
%c  char          : D
%s  string        : Surya
Amount 12345 paise  ->  Rs 123 and 45 paise  (Rs 123.45)

$ cat inputs/input_invalid.txt
1001 abc 600000 8000000000 1.505 2.7182818 D Surya
$ ./task6_io < inputs/input_invalid.txt
INVALID INPUT: scanf converted only 1 of 8 fields -> rejecting

--- wrong format specifiers: compile with -Wall ---
$ gcc -Wall -Wformat -o task6_wrong task6_wrong.c
6:46: warning: format '%d' expects argument of type 'int', but argument 2 has type 'double' [-Wformat=]
7:46: warning: format '%f' expects argument of type 'double', but argument 2 has type 'int' [-Wformat=]
8:46: warning: format '%d' expects argument of type 'int', but argument 2 has type 'long long int' [-Wformat=]
$ ./task6_wrong
printf("%d", 3.14)        -> 759547992
printf("%f", 42)          -> 3.140000
printf("%d", 5000000000LL)-> 705032704   (high bits lost)
printf("%u", -1)          -> 4294967295
printf("%c", 321)         -> A   (321 mod 256 = 65)

--- scanf vs fgets+sscanf vs strtol ---
$ ./task6_compare < inputs/compare_lines.txt
input line     | sscanf("%d") (ret,val)     | strtol (value, leftover, errno)
12abc          | ret=1 val=12          | val=12          TRAILING-JUNK errno=0
abc            | ret=0 val=0           | val=0           NO-DIGITS errno=0
   42          | ret=1 val=42          | val=42          clean errno=0
99999999999    | ret=1 val=1215752191  | val=99999999999 clean errno=0
-7             | ret=1 val=-7          | val=-7          clean errno=0
```

## Task 7 - conversions and precision

```
$ gcc -Wall -Wextra -o task7_conv task7_conv.c -lm   (compiler warnings below are intentional)
14:85: warning: comparison of integer expressions of different signedness: 'int' and 'unsigned int' [-Wsign-compare]
38:105: warning: comparison of integer expressions of different signedness: 'long unsigned int' and 'int' [-Wsign-compare]
$ ./task7_conv
== 1. Integer promotion ==
a + b = 120000 (both promoted to int, so NO 16-bit wrap); (unsigned short)(a+b) = 54464
char 100 + char 100 = 200 (promoted to int; not -56)

== 2. Usual arithmetic conversions (signed vs unsigned) ==
(-1 < 1u) is 0   <- UNEXPECTED: -1 is converted to 4294967295u

== 3. Integer division errors ==
7 / 2 = 3,  7 / 2.0 = 3.5,  1 / 3 * 3.0 = 0.0 (UNEXPECTED: 0)
avg paise/unit: total/units = 210 (integer, truncated) vs (double) = 210.00
(units/100)*100 = 200  (not 250) <- slab rounding bug

== 4. Explicit casts and narrowing ==
(int)5000000000L = 705032704  (truncated; sizeof(long)=8, sizeof(int)=4)
(unsigned char)300 = 44,  (int)3.99 = 3,  (int)-3.99 = -3

== 5. Rounding fractional charge to paise: 33 units x Rs 1.505 ==
rupees (double)        = 49.664999999999915
(long)(rupees*100)     = 4966   <- truncation (wrong)
lround(rupees*100)     = 4967   <- right here only because rupees*100 rounds to 4966.5; not guaranteed
integer: (33*1505+5)/10= 4967   <- exact (rate kept as 1505 milli-rupees)

== 6. Other unexpected expression results ==
0.1 + 0.2 == 0.3 ? 0
float: ten times 0.1f = 1.00000012, == 1.0f ? 0
sizeof(char) > -1 ? 0  (sizeof is size_t, unsigned: -1 becomes huge -> 0)
5 / 2 * 2.0 = 4.0  but 5 * 2.0 / 2 = 5.0

== 7. Accuracy: float vs integer paise (1,000,000 x Rs 0.07) ==
exact = Rs 70000.00 | float = 70266.8750 (err +266.8750) | double = 70000.000000 (err +0.00000019) | paise = 70000.00
Space: float 4 B, double 8 B, long 8 B (int32 4 B).  Time: integer add is 1 cycle; FP add 3-5 cycles latency.
```

## Task 8 - compilation stages, errors, warnings

```
===== STAGE 1: PREPROCESSING  (gcc -E) =====
$ gcc -E main.c -o stages/main.i
$ wc -l main.c stages/main.i
   38 main.c
 1220 stages/main.i
 1258 total
$ grep -c "^#define" stages/main.i   (macros are gone after preprocessing)
0
$ grep -n "RATE\|DBG\|TARIFF_H" stages/main.i | head -3   (no macro names remain)
(no output = none left)
$ tail -n 6 stages/main.i   (main() body: #include expanded, comments removed)
        grand += b.total_paise; ++ok;
    }
    printf("\nValid bills: %d   Rejected: %d   Grand total: ", ok, bad);
    print_money(grand); printf("  (%ld paise)\n", grand);
    return 0;
}
```

```
===== STAGE 2: COMPILATION TO ASSEMBLY (gcc -S) =====
$ gcc -S main.c -o stages/main.s
$ head -n 12 stages/main.s
    .file    "main.c"
    .text
    .section    .rodata
.LC0:
    .string    "Units"
.LC1:
    .string    "Reading"
.LC2:
    .string    "Category"
.LC3:
    .string    "HH"
.LC4:

===== STAGE 3: ASSEMBLY TO OBJECT (gcc -c) =====
$ gcc -c main.c -o stages/main.o ; gcc -c billing.c -o stages/billing.o
$ file stages/main.o
stages/main.o: ELF 64-bit LSB relocatable, x86-64, version 1 (SYSV), not stripped
$ nm stages/main.o | grep " U "   (U = undefined, resolved only at link time)
                 U CATEGORY_NAME
                 U __isoc99_sscanf
                 U __stack_chk_fail
                 U fgets
                 U generate_bill
                 U parse_reading
                 U print_money
                 U printf
                 U puts
                 U status_text
                 U stdin
$ nm stages/billing.o | grep " T "   (T = functions defined here)
000000000000016a T compute_units
00000000000001e2 T energy_charge_paise
000000000000024e T energy_charge_paise_array
00000000000002f8 T generate_bill
0000000000000000 T parse_reading
000000000000046b T print_money
00000000000003e5 T status_text

===== STAGE 4: LINKING =====
$ gcc stages/main.o stages/billing.o -o stages/billing_linked
linked OK
$ ./stages/billing_linked < inputs/five_bills.txt | tail -3
5      2005  Agriculture  2000->2250       250        50000       2500        52500   (Rs 525.00)

Valid bills: 5   Rejected: 0   Grand total: Rs 1990.00  (199000 paise)

===== COMPILE-TIME vs LINK-TIME vs RUNTIME ERRORS =====
$ gcc -o err_compile err_compile.c
err_compile.c: In function 'main':
err_compile.c:5:5: error: expected ',' or ';' before 'printf'
    5 |     printf("%d\n", units);
      |     ^~~~~~
$ gcc -o err_link err_link.c
/usr/bin/ld: /tmp/ccORiaI1.o: in function `main':
err_link.c:(.text+0xe): undefined reference to `energy_charge_paise'
collect2: error: ld returned 1 exit status
$ gcc -o err_runtime err_runtime.c && ./err_runtime
./run_task8.sh: line 40:   379 Floating point exception./err_runtime
exit status = 136 (136 = 128 + SIGFPE)

===== WARNINGS: gcc -Wall -Wextra -O2 warn.c =====
$ gcc -Wall -Wextra -O2 -c warn.c
7:19: warning: comparison of integer expressions of different signedness: 'size_t' {aka 'long unsigned int'} and 'int' [-Wsign-compare]
6:9: warning: unused variable 'unused_var' [-Wunused-variable]
2:45: warning: unused parameter 'limit' [-Wunused-parameter]
8:30: warning: 'c' may be used uninitialized [-Wmaybe-uninitialized]
8:30: warning: 'c' is used uninitialized [-Wuninitialized]
--- after fixing (warn_fixed.c) ---
$ gcc -Wall -Wextra -O2 -o warn_fixed warn_fixed.c
(no warnings)
2

===== SINGLE-FILE vs SEPARATE COMPILATION =====
$ time gcc -o billing_single main.c billing.c   (everything recompiled)
   real 0.080s
$ gcc -c billing.c ; gcc -c main.c ; gcc main.o billing.o   (initial separate build)
(edit only main.c ...) $ time { gcc -c main.c ; gcc main.o billing.o }   (billing.o reused)
   real 0.049s
```

### Task 9 - tests (normal, boundary, rollover, invalid)

```
$ gcc -Wall -Wextra -o billing main.c billing.c
(compiled with no warnings)
$ ./billing < inputs/tests_input.txt
Line   HH    Category    Reading       Units       Energy      Fixed        Total
3      1001  Domestic     1000->1050        50         7500       5000        12500   (Rs 125.00)
4      1002  Domestic     1000->1150       150        25000       5000        30000   (Rs 300.00)
5      1003  Domestic     1000->1300       300        65000       5000        70000   (Rs 700.00)
6      1004  Commercial    500->620        120        19000      15000        34000   (Rs 340.00)
7      1005  Agriculture  2000->2250       250        50000       2500        52500   (Rs 525.00)
9      1006  Domestic      500->600        100        15000       5000        20000   (Rs 200.00)
10     1007  Domestic      500->700        200        35000       5000        40000   (Rs 400.00)
11     1008  Domestic      500->500          0            0       5000         5000   (Rs 50.00)
```

```
12     1009  Domestic      500->601        101        15200       5000        20200   (Rs 202.00)
13     1010  Domestic      500->701        201        35300       5000        40300   (Rs 403.00)
15     1011  Domestic    65535->0            1          150       5000         5150   (Rs 51.50)
16     1012  Domestic    65500->40          76        11400       5000        16400   (Rs 164.00)
17     1013  Domestic    65400->100        236        45800       5000        50800   (Rs 508.00)
19     ERROR (HH 1014): reading is not a number (letters/symbols)  [prev=abc curr=100]
20     ERROR (HH 1015): reading is not a number (letters/symbols)  [prev=100 curr=12x]
21     ERROR (HH 1016): negative reading  [prev=-5 curr=100]
22     ERROR (HH 1017): current < previous without rollover  [prev=1200 curr=1100 rollover=n]
23     ERROR (HH 1018): reading outside 0..65535  [prev=100 curr=70000]
24     ERROR (HH 1019): rollover flagged but current >= previous  [prev=100 curr=200 rollover=y]
25     ERROR (HH 1020): invalid category (0=Dom,1=Com,2=Agri)  [prev=100 curr=200 rollover=n]
26     ERROR: malformed line -> bad line here

Valid bills: 13   Rejected: 8   Grand total: Rs 3968.50  (396850 paise)
```

### Task 9 - slabs vs array, five bills

```
$ gcc -Wall -Wextra -o test_slabs test_slabs.c billing.c -O2
$ ./test_slabs
Compared if-else vs array/loop for units 0..1000 -> mismatches = 0
units=0    expected energy=0      if-else=0      array=0      PASS
units=100  expected energy=15000  if-else=15000  array=15000  PASS
units=101  expected energy=15200  if-else=15200  array=15200  PASS
units=200  expected energy=35000  if-else=35000  array=35000  PASS
units=201  expected energy=35300  if-else=35300  array=35300  PASS
Time for 20,000,000 bill calculations: if-else = 0.037 s, array+loop = 0.071 s

$ ./billing < inputs/five_bills.txt   (five bills, compare with manual table)
Line   HH    Category    Reading        Units       Energy      Fixed        Total
1      2001  Domestic     1000->1050        50         7500       5000        12500   (Rs 125.00)
2      2002  Domestic     1000->1150       150        25000       5000        30000   (Rs 300.00)
3      2003  Domestic     1000->1300       300        65000       5000        70000   (Rs 700.00)
4      2004  Commercial    500->620        120        19000      15000        34000   (Rs 340.00)
5      2005  Agriculture  2000->2250       250        50000       2500        52500   (Rs 525.00)

Valid bills: 5   Rejected: 0   Grand total: Rs 1990.00  (199000 paise)
```

### Task 10 - district scale

```
$ gcc -Wall -Wextra -o task10_scale task10_scale.c billing.c -O2
$ ./task10_scale
Bills     exact (int64 paise) float rupees         double rupees      int32 total    long total
4000      Rs 2194780          2194780.00           2194780.0000       ok             219478000
          exact = 219478000 paise | float error = +0.00 Rs | double error = +0.000000 Rs | int32 wrapped value = 219478000
60000     Rs 32924559         32923508.00          32924559.0000      OVERFLOW       3292455900
          exact = 3292455900 paise | float error = -1051.00 Rs | double error = +0.000000 Rs | int32 wrapped value = -1002511396
800000    Rs 439021194        439074240.00         439021194.0000     OVERFLOW       43902119400
          exact = 43902119400 paise | float error = +53046.00 Rs | double error = +0.000000 Rs | int32 wrapped value = 952446440

-- long long vs double for district totals (exactness limit) --
2^53+1 as long long = 9007199254740993 ; stored in double = 9007199254740992  (double cannot hold it exactly)
Largest district total (800,000 bills x 12 months x Rs 50,000 max) = 480000000000000 paise (< 2^53 = 9007199254740992, so both fit)
Max values: INT_MAX=2147483647 (Rs 21474836.47), LONG_MAX=9223372036854775807, LLONG_MAX=9223372036854775807

-- Actual vs minimum guaranteed by C standard --
type         this system  C standard min actual range
int          4            2 B (16 bit)   -2147483648 .. 2147483647
long         8            4 B (32 bit)   -9223372036854775808 .. 9223372036854775807
long long    8            8 B (64 bit)   -9223372036854775808 .. 9223372036854775807
unsigned short 2            2 B (16 bit)   0 .. 65535
Standard guarantees only: int >= -32767..32767, long >= +-2147483647, long long >= +-9223372036854775807

-- Time: 50,000,000 additions --
double = 0.144 s   long long = 0.018 s   (both 8 bytes of space)
```

### Task 12 - smart meter

```
$ gcc -Wall -Wextra -o smart_meter smart_meter.c
$ ./smart_meter tariff.cfg
Loaded tariff from tariff.cfg: slabs 100000/200000 Wh, rates 150/200/300 paise/kWh, peak 18:00-22:00 +20%, fixed 5000 paise
Readings per day = 96, per year = 35040 (365 x 96)

-- Monthly bill (30 days = 2880 readings) --
Total consumption : 264000 Wh (264.000 kWh)   [16-bit register would wrap 4 times]
Peak consumption  : 120000 Wh   Off-peak: 144000 Wh
Energy charge     : 59158 paise   Fixed: 5000 paise   TOTAL: 64158 paise = Rs 641.58
Same month without peak rule: 59200 paise -> peak rule adds 4958 paise

-- Largest possible values (10 kW max load, 800000 households) --
Max Wh per 15-min reading      : 2500   (16-bit max 65535 -> fits)
Max Wh per household per year  : 87600000   (uint32 max 4294967295 -> fits uint32, NOT 16-bit)
Max Wh for whole district/year : 70080000000000   (uint32 max 4294967295 -> needs 64-bit)
Max intermediate paise*Wh*100  : 3153600000000   (int32 max 2147483647 -> needs 64-bit)
INT64_MAX                      : 9223372036854775807

--- change tariff WITHOUT recompiling: RATE3 300 -> 350 in tariff2.cfg ---
$ ./smart_meter tariff2.cfg
Loaded tariff from tariff2.cfg: slabs 100000/200000 Wh, rates 150/200/350 paise/kWh, peak 18:00-22:00 +20%, fixed 5000 paise
Energy charge     : 62658 paise   Fixed: 5000 paise   TOTAL: 67658 paise = Rs 676.58
```

# Section H - Build scripts

### run_all.sh - compiles and runs everything

```
#!/bin/bash
# Compile and run every task; all output is stored in outputs/
cd "$(dirname "$0")"; O=outputs; mkdir -p $O
W="-Wall -Wextra"
{
echo '$ gcc -Wall -Wextra -o billing main.c billing.c'; gcc $W -o billing main.c billing.c && echo '(compiled with no warnings)'
echo '$ ./billing < inputs/tests_input.txt'; ./billing < inputs/tests_input.txt
} > $O/task9_tests.txt 2>&1
{ echo '$ gcc -Wall -Wextra -o test_slabs test_slabs.c billing.c -O2'; gcc $W -O2 -o test_slabs test_slabs.c billing.c; echo '$ ./test_sla\
    bs'; ./test_slabs
  echo; echo '$ ./billing < inputs/five_bills.txt   (five bills, compare with manual table)'; ./billing < inputs/five_bills.txt; } > $O/ta\
    sk9_slabs.txt 2>&1
{ echo '$ gcc -Wall -Wextra -o task2_types task2_types.c'; gcc $W -O0 -o task2_types task2_types.c; echo '$ ./task2_types'; ./task2_types;\
     } > $O/task2.txt 2>&1
{ echo '--- Run 1: original RATE2_PAISE = 200 ---'; echo '$ gcc -Wall -Wextra -o task3 task3_const.c billing.c'; gcc $W -o task3 task3_con\
    st.c billing.c; echo '$ ./task3'; ./task3
  echo; echo '--- Run 2: RATE2_PAISE changed to 230 (no source edit: -DRATE2_PAISE=230) ---'; echo '$ gcc -Wall -Wextra -DRATE2_PAISE=230 \
    -o task3_r230 task3_const.c billing.c'; gcc $W -DRATE2_PAISE=230 -o task3_r230 task3_const.c billing.c; echo '$ ./task3_r230'; ./task3\
    _r230
  echo; echo '--- Five bills re-run with RATE2_PAISE=230 ---'; gcc $W -DRATE2_PAISE=230 -o billing_r230 main.c billing.c; ./billing_r230 <\
     inputs/five_bills.txt; } > $O/task3.txt 2>&1
{ echo '$ gcc -Wall -Wextra -O0 -o task4_macros task4_macros.c'; gcc $W -O0 -o task4_macros task4_macros.c; echo '$ ./task4_macros   (-O0)\
    '; ./task4_macros
  echo; echo '$ gcc -O2 -o task4_macros_O2 task4_macros.c ; ./task4_macros_O2   (-O2)'; gcc -O2 -o task4_macros_O2 task4_macros.c; ./task4\
    _macros_O2 | tail -3; } > $O/task4_run.txt 2>&1
{ echo '$ cat macro_expand.c'; cat macro_expand.c; echo; echo '$ gcc -E macro_expand.c | tail -n 4'; gcc -E macro_expand.c | tail -n 4; } \
    > $O/task4_expand.txt 2>&1
{ echo '$ gcc -Wall -Wextra -o t5_nodebug task5_debug.c billing.c'; gcc $W -o t5_nodebug task5_debug.c billing.c; echo '$ ./t5_nodebug'; .\
    /t5_nodebug
  echo; echo '$ gcc -Wall -Wextra -DDEBUG -o t5_debug task5_debug.c billing.c'; gcc $W -DDEBUG -o t5_debug task5_debug.c billing.c; echo '\
    $ ./t5_debug'; ./t5_debug
  echo; echo '$ ./t5_nodebug --debug   (runtime debugging via command-line flag)'; ./t5_nodebug --debug
  echo; echo '$ ls -l t5_nodebug t5_debug   (executable size, bytes)'; ls -l t5_nodebug t5_debug | awk '{print $5, $9}'
  echo; echo '$ size t5_nodebug t5_debug'; size t5_nodebug t5_debug
  echo; echo '--- include guard: header WITHOUT guard included twice ---'; echo '$ gcc noguard_demo.c'; gcc noguard_demo.c 2>&1 | head -8
  echo; echo '--- #include <file> vs #include "file" ---'; printf '#include <tariff.h>\nint main(void){return 0;}\n' > angle.c
  echo '$ gcc -c angle.c            (angle brackets: system paths only)'; gcc -c angle.c 2>&1 | head -4
  echo '$ gcc -I. -c angle.c        (-I. adds current dir to the <> search path)'; gcc -I. -c angle.c && echo 'compiled OK'
  printf '#include "tariff.h"\nint main(void){return 0;}\n' > quote.c; echo '$ gcc -c quote.c            (quotes: current dir first, then \
    system paths)'; gcc -c quote.c && echo 'compiled OK'
  echo; echo '--- system include search path (excerpt) ---'; echo | gcc -E -Wp,-v -xc - 2>&1 | sed -n '/#include <...> search/,/End of sea\
    rch/p'; } > $O/task5.txt 2>&1
{ echo '$ gcc -Wall -Wextra -o task6_io task6_io.c'; gcc $W -o task6_io task6_io.c
  echo '$ cat inputs/input_valid.txt'; cat inputs/input_valid.txt; echo '$ ./task6_io < inputs/input_valid.txt'; ./task6_io < inputs/input\
    _valid.txt
  echo; echo '$ cat inputs/input_invalid.txt'; cat inputs/input_invalid.txt; echo '$ ./task6_io < inputs/input_invalid.txt'; ./task6_io < \
    inputs/input_invalid.txt
  echo; echo '--- wrong format specifiers: compile with -Wall ---'; echo '$ gcc -Wall -Wformat -o task6_wrong task6_wrong.c'; gcc -Wall -W\
    format -o task6_wrong task6_wrong.c 2>&1 | grep -E 'warning' | sed 's/^task6_wrong.c://'
  echo '$ ./task6_wrong'; ./task6_wrong
  echo; echo '--- scanf vs fgets+sscanf vs strtol ---'; gcc $W -o task6_compare task6_compare.c; echo '$ ./task6_compare < inputs/compare_\
    lines.txt'; ./task6_compare < inputs/compare_lines.txt; } > $O/task6.txt 2>&1
{ echo '$ gcc -Wall -Wextra -o task7_conv task7_conv.c -lm   (compiler warnings below are intentional)'; gcc $W -o task7_conv task7_conv.c\
     -lm 2>&1 | grep warning | sed 's/^task7_conv.c://'; echo '$ ./task7_conv'; ./task7_conv; } > $O/task7.txt 2>&1
{ echo '$ gcc -Wall -Wextra -o task10_scale task10_scale.c billing.c -O2'; gcc $W -O2 -o task10_scale task10_scale.c billing.c; echo '$ ./\
    task10_scale'; ./task10_scale; } > $O/task10.txt 2>&1
{ echo '$ gcc -Wall -Wextra -o smart_meter smart_meter.c'; gcc $W -o smart_meter smart_meter.c; echo '$ ./smart_meter tariff.cfg'; ./smart\
    _meter tariff.cfg
  echo; echo '--- change tariff WITHOUT recompiling: RATE3 300 -> 350 in tariff2.cfg ---'; sed 's/^RATE3_PAISE_PER_KWH .*/RATE3_PAISE_PER_\
    KWH       350/' tariff.cfg > tariff2.cfg; echo '$ ./smart_meter tariff2.cfg'; ./smart_meter tariff2.cfg | sed -n '1p;/Energy charge/p\
    '; } > $O/task12.txt 2>&1
echo done
```